

AI QUIZ GENERATOR

Multi-Step Generative AI Workflow for Automatic MCQ Generation, Evaluation & Explanations

STUDENT	Meduri Bhargav Rakesh
REGISTER NO.	99230040667
DATE	06-09-2026
ASSESSMENT	Mini Assessment — GenAI Workflow Design

01 | Concept

Purpose: The AI Quiz Generator is a simple Generative AI application that creates multiple-choice questions (MCQs) from a topic entered by the user. The system generates questions, four answer options, the correct answer, and a short explanation for each question.

Problem: Creating practice questions manually takes time and may not provide enough variety or appropriate difficulty. The AI Quiz Generator automatically converts a topic into an interactive quiz for learning and self-assessment.

02 | Objective

The objective is to design a prompt-based GenAI workflow that accepts a topic and difficulty level, generates MCQs, evaluates the user's answers, calculates the score, and provides explanations for the answers.

03 | Workflow Overview

1. Input	2. Generate	3. Attempt	4. Evaluate	5. Explain
Topic	MCQs	Answers	Score	Feedback

04 | Workflow Steps

- 1. Topic & Preference Input:** Collect the topic, number of questions, and difficulty level selected by the user.
- 2. Question Generation:** Use a prompt template to generate clear MCQs with four options and one correct answer.
- 3. Quiz Presentation:** Display the generated questions in an interactive interface so the user can select answers.
- 4. Answer Evaluation:** Compare the user's selected answers with the generated answer key and calculate the total score.
- 5. Explanation Generation:** Provide a concise explanation for each answer so the user understands why the answer is correct or incorrect.

05 | Prompt Workflow

Step 1 — Understand User Input

Read the topic, number of questions, and selected difficulty.

Step 2 — Generate Questions

Create questions relevant to the selected topic and difficulty.

Step 3 — Generate Answer Key

For each question, identify one correct answer and plausible distractors.

Step 4 — Evaluate User Answers

Compare submitted answers with the answer key and calculate the score.

Step 5 — Generate Feedback

Explain the correct answer and clarify mistakes to support learning.

06 | Prompt Chaining Architecture

STAGE	INPUT / CONTEXT	OUTPUT
1	Topic + difficulty + question count	Quiz requirements
2	Quiz requirements	Generated MCQs
3	MCQs + answer key	Interactive quiz
4	User responses + answer key	Score + question results
5	Question results	Explanations + learning feedback
6	Score + weak areas	Suggested revision topics

07 | GenAI Components

- Prompt Templates — keep question-generation instructions consistent while inserting dynamic topics and difficulty levels.
- LLM — generates questions, options, answer keys, explanations, and revision suggestions.
- Structured Output — stores question, options, correct answer, and explanation in a predictable format.
- Prompt Chaining — connects question generation, answer evaluation, and feedback generation.
- Prompt Evaluation — tests generated quizzes for relevance, correctness, uniqueness, and difficulty.

08 | Demonstration Input

Example Scenario: The user enters the topic **Machine Learning**, selects **Medium** difficulty, and requests **5 questions**. The AI generates five MCQs covering supervised learning, classification, regression, training data, and model evaluation.

09 | Expected Output

The application displays five MCQs with four options each. After the user submits the quiz, the system checks the responses, calculates the score, identifies incorrect answers, and displays explanations for each question.

INPUT	Machine Learning + Medium + 5 questions
GENERATION	5 MCQs + 4 options + answer key
USER ACTION	Select one answer for each question
EVALUATION	Compare responses with answer key
FINAL OUTPUT	Score + correct answers + explanations + revision suggestions

10 | Sample Support Execution Plan

PHASE	ACTION FOCUS	OUTPUT / METRIC
Phase 1	Requirement Collection	Topic → quiz requirements
Phase 2	Question Generation	Requirements → MCQs
Phase 3	Quiz Interface	MCQs → user responses
Phase 4	Evaluation	Responses → score
Phase 5	Feedback	Results → explanations
Phase 6	Revision	Weak areas → suggested topics

11 | Key Benefits

- ✓ Automatically creates practice questions from any selected topic
- ✓ Supports different difficulty levels
- ✓ Saves time compared with manually creating quizzes
- ✓ Provides instant scoring and explanations
- ✓ Helps identify weak areas for revision
- ✓ Demonstrates Generative AI, prompt engineering, structured output, and prompt chaining

12 | Discussion Questions & Answers

1. What is the advantage of using {user_input} instead of hard-coding input?

It allows the same quiz-generation prompt to work with different topics, difficulty levels, and question counts.

2. When would .format() be enough?

Python's .format() is enough when the prompt only needs simple values such as a topic, difficulty, or number of questions.

3. When might Jinja2 be useful?

Jinja2 is useful when prompts contain conditional sections, optional instructions, or different templates.

4. Why should prompts be versioned?

Prompt versioning helps track changes and compare which prompt produces more accurate and relevant quiz questions.

5. What is a prompt regression?

A prompt regression occurs when a prompt modification improves one quiz scenario but causes another to become less accurate or consistent.

6. Why use the same evaluation set when comparing prompt versions?

The same test topics and requirements make prompt comparison fair and measurable.

13 | Suggested Technology Stack

LAYER	TECHNOLOGY OPTIONS
Language	Python
LLM	OpenAI / Gemini / other course-approved LLM
Prompt Framework	LangChain or direct API integration
Interface	Streamlit
Data Storage	JSON / SQLite (optional)
Output	Interactive quiz + score + explanations
Evaluation	Fixed quiz scenarios + quality rubric